

Pipeline for logP Model Deployment on ESP32

Aspirin is shown as an example input; the neural network is trained on the training subset of the dataset.

1. Canonicalize the Input

RDKit standardizes the example before feature generation.

Input SMILES

CC(=O)OC1=CC=CC=C1C(=O)O

RDKit
cleanup +
canonicalization

Canonical SMILES

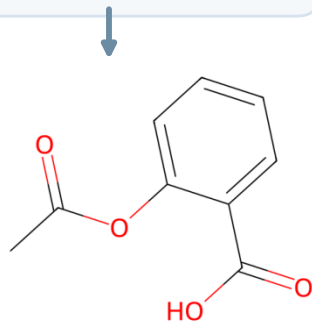
CC(=O)Oc1ccccc1C(=O)O

2. Build the Molecule Graph

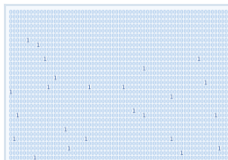
Canonical SMILES are parsed into an RDKit Mol, then fingerprinted with Morgan/ECFP4.

Canonical SMILES

CC(=O)Oc1ccccc1C(=O)O



Morgan
radius = 2
2048 bits

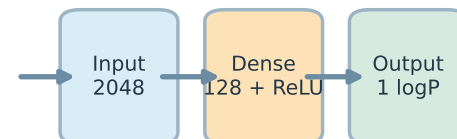
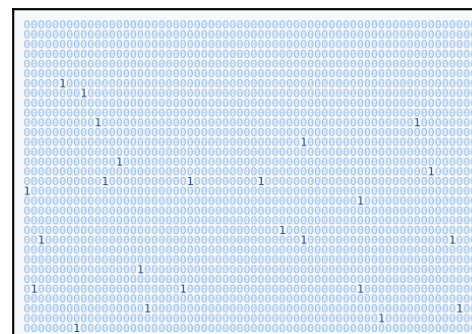


Canonical SMILES -> RDKit Mol object -> Morgan fingerprint.

3. Train the Neural Network

Morgan fingerprints become the model input during supervised training.

Generated ECFP4 fingerprint



Dataset split: train 80% | validation 10% | test 10%

Hyperparameters: batch size 256, up to 20 epochs, Adam optimizer, MSE loss

4. Quantize the Model

Post-training LiteRT conversion compresses the model for edge deployment.

**Keras
model**

LiteRT
converter

**Quantized
.tflite**

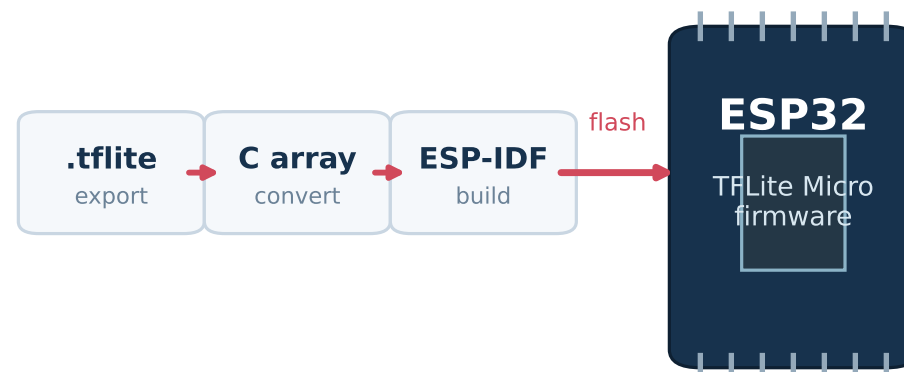
Quantization
PTQ int8

Model size
261.5 KiB

Tensor types
in: (1, 2048), int8
out: (1, 1), float32

5. Flash to the ESP32

The quantized model is converted into firmware assets and flashed with ESP-IDF.



USB/UART flashing via the ESP-IDF VS Code Extension